



資料結構

Data Structure

Lab 08

姓名：夏宇柔
學號：113840005

Lab08-Q1.

Modify the inorder traversal function to a postorder traversal function in Part1.

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;           // 節點的值
    TreeNode* left;      // 左子節點
    TreeNode* right;     // 右子節點

    // constructor: initializes node with value and sets children to nullptr
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {}
};

// 樹結構
class BinaryTree {
public:
    TreeNode* root;

    // constructor: initialize tree with empty root
    BinaryTree() : root(nullptr) {}

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(root); // 將根節點加入queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出隊列中的節點
            q.pop();

            // 添加左子節點
            if (i < arr.size()) {
                current->left = new TreeNode(arr[i]);
                q.push(current->left); // 將左子節點加入queue
                i++;
            }

            // 添加右子節點
            if (i < arr.size()) {
                current->right = new TreeNode(arr[i]);
                q.push(current->right); // 將右子節點加入queue
                i++;
            }
        }
    }
}
```

```

        return root;
    }

    // 中序遍歷
    void inorderTraversal(TreeNode* node) {
        if (node == nullptr) return; // 如果節點為空，忽略

        inorderTraversal(node->left); // 遍歷左子樹
        cout << node->value << " "; // 訪問當前節點
        inorderTraversal(node->right); // 遍歷右子樹
    }

    // postorder traversal
    void postorderTraversal(TreeNode* node) {
        if (node == nullptr) return; // stop at null node

        postorderTraversal(node->left); // first left subtree
        postorderTraversal(node->right); // then right subtree
        cout << node->value << " "; // last the node itself
    }
};

int main() {
    BinaryTree tree; // 宣告二元樹

    // 輸入陣列用於構建樹，NULL 表示空子節點
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

    tree.buildTree(arr); // 建立樹

    // 中序遍歷輸出
    cout << "Inorder Traversal: ";
    tree.inorderTraversal(tree.root);
    cout << endl;

    // output: postorder traversal
    cout << "Postorder Traversal: ";
    tree.postorderTraversal(tree.root);

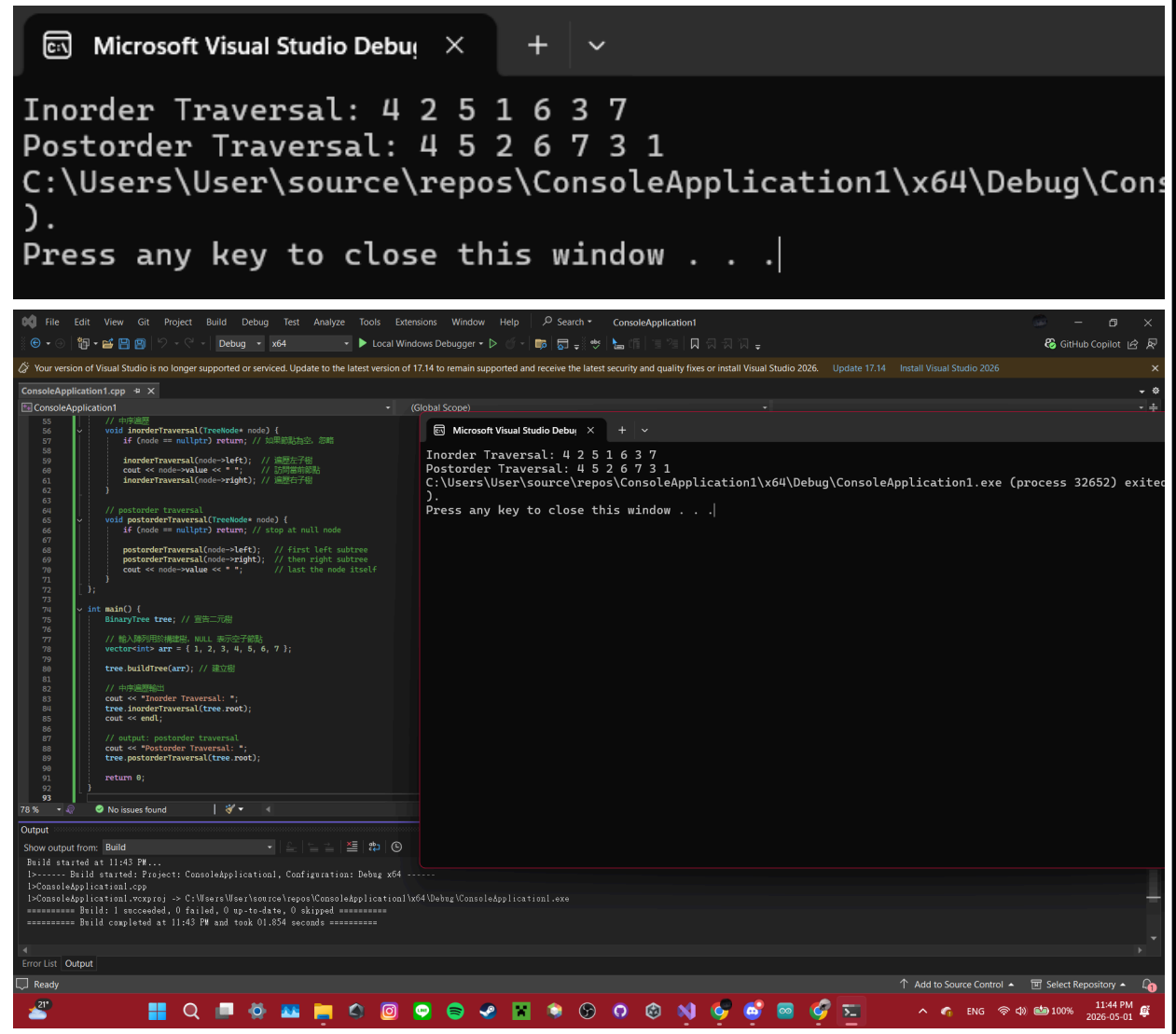
    return 0;
}

```

Result

Inorder Traversal: 4 2 5 1 6 3 7

Postorder Traversal: 4 5 2 6 7 3 1



Lab08-Q2.

Implement a method to find the maximum value in the left and right subtrees.

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;           // 節點的值
    TreeNode* left;      // 左子節點
    TreeNode* right;     // 右子節點

    // constructor: initializes node with value and sets children to nullptr
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {}
};

// 樹結構
class BinaryTree {
public:
    TreeNode* root;

    // constructor: initialize tree with empty root
    BinaryTree() : root(nullptr) {}

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(root); // 將根節點加入queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出隊列中的節點
            q.pop();

            // 添加左子節點
            if (i < arr.size()) {
                current->left = new TreeNode(arr[i]);
                q.push(current->left); // 將左子節點加入queue
                i++;
            }

            // 添加右子節點
            if (i < arr.size()) {
                current->right = new TreeNode(arr[i]);
                q.push(current->right); // 將右子節點加入queue
                i++;
            }
        }

        return root;
    }
};
```

```

}

// 中序遍歷
void inorderTraversal(TreeNode* node) {
    if (node == nullptr) return; // 如果節點為空, 忽略

    inorderTraversal(node->left); // 遍歷左子樹
    cout << node->value << " "; // 訪問當前節點
    inorderTraversal(node->right); // 遍歷右子樹
}

// recursive to find max value in a subtree
int findMax(TreeNode* node) {
    if (node == nullptr) return INT_MIN;
    // if node is null, return smallest int so it won't affect max
    // comparison

    // recursive to find max in left and right subtrees
    int leftMax = findMax(node->left);
    int rightMax = findMax(node->right);

    // return the largest in current node, left subtree, and right
    // subtree
    return max(node->value, max(leftMax, rightMax));
}

// print max values in left and right subtrees of root
void printLeftRightMax() {
    if (root == nullptr) {
        cout << "Tree is empty." << endl;
        return;
    }

    // find max in left subtree
    int leftMax = findMax(root->left);

    // find max in right subtree
    int rightMax = findMax(root->right);

    cout << "Max value in left subtree: " << leftMax << endl;
    cout << "Max value in right subtree: " << rightMax << endl;
}

};

int main() {
    BinaryTree tree; // 宣告二元樹

    // 輸入陣列用於構建樹, NULL 表示空子節點
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

    tree.buildTree(arr); // 建立樹

    // 中序遍歷輸出
    cout << "Inorder Traversal: ";
    tree.inorderTraversal(tree.root);
    cout << endl;
}

```

```

        // output: max values in left and right subtrees
        tree.printLeftRightMax();

        return 0;
    }
}

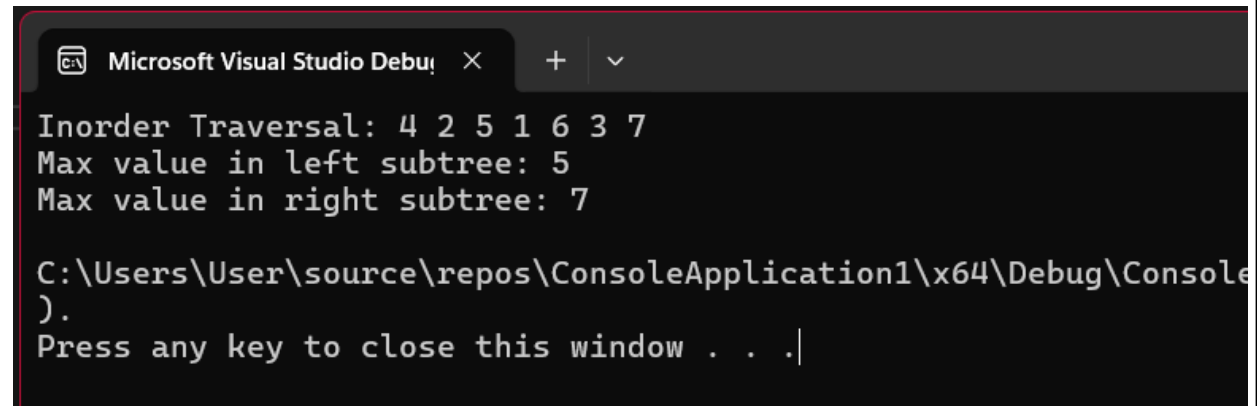
```

Result

Inorder Traversal: 4 2 5 1 6 3 7

Max value in left subtree: 5

Max value in right subtree: 7

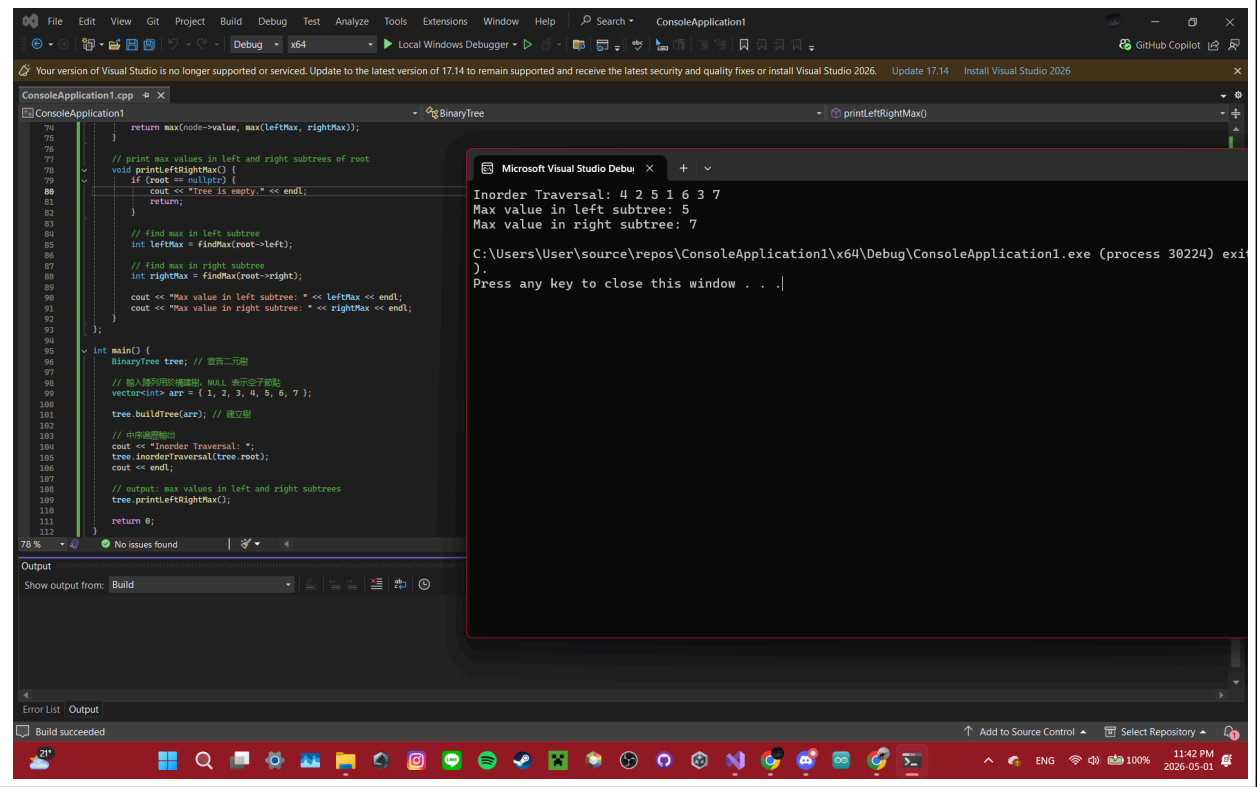


```

Microsoft Visual Studio Debug Console
Inorder Traversal: 4 2 5 1 6 3 7
Max value in left subtree: 5
Max value in right subtree: 7

C:\Users\User\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe
Press any key to close this window . . .|

```



```

// ConsoleApplication1.cpp
74         return max(node->value, max(leftMax, rightMax));
75     }
76
77     // print max values in left and right subtrees of root
78     void printLeftRightMax() {
79         if (root == nullptr) {
80             cout << "Tree is empty." << endl;
81             return;
82         }
83
84         // find max in left subtree
85         int leftMax = findMax(root->left);
86
87         // find max in right subtree
88         int rightMax = findMax(root->right);
89
90         cout << "Max value in left subtree: " << leftMax << endl;
91         cout << "Max value in right subtree: " << rightMax << endl;
92     }
93
94
95     int main() {
96         BinaryTreeNode tree; // 定义二叉树
97
98         // 输入用于构造二叉树的序列，NULL 表示空子树
99         vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };
100
101         tree.buildTree(arr); // 建立树
102
103         // 中序遍历输出
104         cout << "Inorder Traversal: ";
105         tree.inorderTraversal(tree.root);
106         cout << endl;
107
108         // output: max values in left and right subtrees
109         tree.printLeftRightMax();
110
111         return 0;
112     }

```

```

Microsoft Visual Studio Debug Console
Inorder Traversal: 4 2 5 1 6 3 7
Max value in left subtree: 5
Max value in right subtree: 7

C:\Users\User\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 38224) exited with code 0.
Press any key to close this window . . .|

```

Lab08-Q3.**Explain how to find the maximum value in the left and right subtrees.****Discussion**

Start from the root, using a recursive function to loop through all of the left subtree and the right subtree. The function does as follows:

- If this node's value is NULL, return the smallest integer possible
- Find the max value of the left subtree
- Find the max value of the right subtree
- Return the max value between this node's value, the max value of the left subtree, and the max value of the right subtree

Using a recursive function ensures all nodes are checked.

Lab08-Q4.**What happens if the left and right subtrees are not complete trees?****Discussion**

The code will still run properly, but may consume more time, due to how deep the recursive function may have to go.